

# Contents

|                                                                      |          |
|----------------------------------------------------------------------|----------|
| □ <b>In-Place Linked List Reversal — Complete Interview Handbook</b> | <b>1</b> |
| Table of Contents . . . . .                                          | 1        |
| SECTION 1 — Pattern Overview . . . . .                               | 2        |
| SECTION 2 — Recognition Guide . . . . .                              | 3        |
| SECTION 3 — Decision Framework . . . . .                             | 3        |
| SECTION 4 — Why This Pattern Works . . . . .                         | 4        |
| SECTION 5 — Algorithm Framework . . . . .                            | 4        |
| SECTION 6 — Time & Space Complexity . . . . .                        | 6        |
| SECTION 7 — Edge Cases . . . . .                                     | 6        |
| SECTION 8 — Pros & Cons . . . . .                                    | 7        |
| SECTION 9 — Real World Applications . . . . .                        | 7        |
| SECTION 10 — Interview Strategy . . . . .                            | 7        |
| SECTION 11 — Pattern Variations . . . . .                            | 8        |
| SECTION 12 — Comparison With Other Patterns . . . . .                | 8        |
| SECTION 13 — Problem Classification . . . . .                        | 8        |
| SECTION 14 — 30 Interview Problems . . . . .                         | 9        |
| SECTION 15 — Common Mistakes . . . . .                               | 11       |
| SECTION 16 — Optimization Techniques . . . . .                       | 12       |
| SECTION 17 — Multiple Language Templates . . . . .                   | 12       |
| SECTION 18 — Dry Runs . . . . .                                      | 14       |
| SECTION 19 — Advanced Concepts . . . . .                             | 14       |
| SECTION 20 — Senior Engineer Insights . . . . .                      | 14       |
| SECTION 21 — Cheat Sheet . . . . .                                   | 14       |
| SECTION 22 — Revision Notes (5-Minute Review) . . . . .              | 15       |
| SECTION 23 — Practice Roadmap . . . . .                              | 15       |
| SECTION 24 — Memory Tricks . . . . .                                 | 15       |
| SECTION 25 — Final Summary . . . . .                                 | 16       |

## □ In-Place Linked List Reversal — Complete Interview Handbook

**Pattern #4 of 28 | DSA Pattern Mastery Series Audience:** Senior/Staff SWE interviews (Google, Amazon, Meta, Microsoft, Uber, Airbnb, Atlassian, Grab, Careem, Noon, Talabat, Dubai/Saudi & India Tier-1/Tier-2 product companies) **Companion:** Pairs with Striver's A2Z DSA Course — Linked List section

---

### Table of Contents

1. Pattern Overview · 2. Recognition Guide · 3. Decision Framework · 4. Why This Pattern Works · 5. Algorithm Framework · 6. Time & Space Complexity · 7. Edge Cases · 8. Pros & Cons · 9. Real World Applications · 10. Interview Strategy · 11. Pattern Variations · 12. Comparison With Other Patterns · 13. Problem Classification · 14. 30 Interview Problems · 15. Common Mistakes · 16. Optimization Techniques · 17. Multiple Language Templates · 18. Dry Runs · 19. Advanced Concepts · 20. Senior Engineer Insights · 21. Cheat Sheet · 22. Revision Notes · 23. Practice Roadmap · 24. Memory Tricks · 25. Final Summary

---

## SECTION 1 — Pattern Overview

### 1.1 What Is This Pattern?

In-Place Linked List Reversal rewires the `next` pointers of a linked list (or a sublist within it) so the traversal direction flips, using only a constant number of extra pointer variables (`prev`, `curr`, `next`) — no auxiliary list, stack, or array is created.

### 1.2 Why Was This Pattern Invented?

The naive way to reverse a list is to push all values onto a stack (or into an array) and rebuild the list —  $O(n)$  extra space. Since a linked list is just a chain of `next` references, you can achieve the same result by **redirecting each node's pointer to its predecessor as you walk forward**, achieving  $O(1)$  space. This generalizes to reversing sublists, groups of  $k$  nodes, and alternating segments.

### 1.3 Real Intuition Behind The Pattern

Imagine a line of people holding hands, each person's right hand holding the next person's left hand. To reverse the line's direction, each person just needs to **switch which hand holds which neighbor** — no one needs to physically move; only the "handshake direction" changes. That's exactly what re-pointing `next` does.

### 1.4 Mental Model

At every step, you have three pointers: `prev` (already-reversed portion's head), `curr` (node being processed), and a saved `next` (to avoid losing the rest of the list once you overwrite `curr.next`). You are always "flipping one link and advancing the frontier by one node."

### 1.5 Visual Explanation

Before: `1 → 2 → 3 → 4 → null`  
`prev=null curr=1`

Step 1: `save next=2; 1.next=null(prev); prev=1; curr=2`  
`null ← 1      2 → 3 → 4 → null`

Step 2: `save next=3; 2.next=1; prev=2; curr=3`  
`null ← 1 ← 2      3 → 4 → null`

... continue until `curr=null`

Final: `null ← 1 ← 2 ← 3 ← 4` (prev is now the new head: 4)

### 1.6 Simple Analogy

Like **turning a one-way street sign around, one segment at a time**, walking from the start to the end of the street and flipping each directional arrow as you pass it — by the time you reach the end, the entire street points the opposite way, and you never needed to build a second street.

### 1.7 When Should I Immediately Think About Using This Pattern?

- "Reverse a linked list" (whole or a sub-range [`left`, `right`]).
- "Reverse nodes in groups of  $k$ ."
- "Check if a linked list is a palindrome" (reverse the second half).
- "Reorder a linked list" (reverse + merge).
- Explicit " **$O(1)$  space**" constraint on any linked-list restructuring problem.

---

## SECTION 2 — Recognition Guide

### 2.1 Keywords

| Keyword                             | Signal                               |
|-------------------------------------|--------------------------------------|
| "reverse the linked list"           | Direct signal                        |
| "reverse between positions m and n" | Sublist reversal variant             |
| "reverse in groups of k"            | Grouped reversal variant             |
| "without using extra space"         | Rules out stack/array-based reversal |
| "swap pairs"                        | Special case of group-of-2 reversal  |

### 2.2 Hidden Hints

A problem defined as "restructure a list" with an  $O(1)$  space constraint almost always decomposes into: find boundary nodes (often via Fast & Slow) → reverse a segment → re-link boundaries.

### 2.3 Interview Clues

Interviewer draws arrows on a whiteboard and asks "how would you flip these without extra memory?" — the explicit visual of arrows is a strong tell.

### 2.4 Common Trick Words

"In groups of k," "alternate every other group," "between positions" — all signal sub-range reversal requiring careful boundary bookkeeping (dummy nodes, saved predecessor/successor pointers).

### 2.5 What Interviewers Expect

Correct pointer bookkeeping (never losing the rest of the list), correct use of a **dummy node** to simplify head-edge-case handling, and a clean recursive alternative articulated alongside the iterative one.

### 2.6 When NOT To Use This Pattern

- You need **random access** reversal or reversal of an array — simple index-swap Two Pointers is simpler and more appropriate there.
- You need to preserve the original list **and** produce a reversed copy — then  $O(n)$  extra space is unavoidable (or required by the problem), so this in-place technique doesn't apply as-is.
- The structure isn't a singly-linked chain (e.g., you need to reverse a tree's structure) — different techniques apply.

---

## SECTION 3 — Decision Framework

Is the data a singly (or doubly) linked list?

Yes

No → use array index-swap Two Pointers instead

Do you need  $O(1)$  space?

Yes → USE IN-PLACE REVERSAL (iterative prev/curr/next)

No → stack/array based reversal is simpler to write, acceptable if space unconstrained

Is it a FULL reversal or a SUB-RANGE / GROUPED reversal?

Full → single pass, prev/curr/next loop

Sub-range/Grouped → locate boundary nodes first (often via counting or Fast/Slow),  
then apply the same core loop within that boundary,  
then re-link to the untouched parts using saved predecessor/successor

**Why:** The core reversal loop is identical in every variant; what changes is **boundary identification and re-linking**, which is where most bugs occur.

---

## SECTION 4 — Why This Pattern Works

**Mathematical/Logical:** Reversal is fundamentally a permutation of next pointers such that if originally  $\text{next}(a) = b$ , after reversal  $\text{next}(b) = a$ . Since each node's next field is a single mutable reference, updating it in a single forward pass — provided you've already saved the reference to what comes *after* the current node before overwriting — loses no information. This is a **classic three-pointer invariant**: at the start of each iteration, prev holds the fully-reversed prefix, curr is the node about to be flipped, and the untouched suffix is still reachable via the pre-saved next reference.

**Intuitive:** You never need to “look back” at what you've already reversed — the direction of causality is one-way, and preserving the “next in original order” reference just long enough to not lose it is the entire trick.

**Correctness Proof:** *Invariant:* after processing  $i$  nodes, prev is the head of a correctly-reversed list of the first  $i$  nodes, and curr points to the  $(i+1)$ th original node, with the rest of the list still intact and reachable from curr. *Base case:*  $i=0$ , prev=null, curr=head — trivially true. *Inductive step:* saving next = curr.next before reassigning curr.next = prev preserves reachability to the remaining  $n-i-1$  nodes; advancing prev=curr, curr=next extends the invariant to  $i+1$ . *Termination:* when curr=null, all  $n$  nodes have been reversed; prev is the new head. **QED.**

---

## SECTION 5 — Algorithm Framework

### 5.1 Step-by-Step Framework (Full Reversal)

1. Initialize prev = null, curr = head.
2. While curr != null: save next = curr.next; set curr.next = prev; advance prev = curr; advance curr = next.
3. Return prev as the new head.

### 5.2 General Template

```
function reverseList(head):  
    prev = null  
    curr = head  
    while curr != null:  
        nextNode = curr.next
```

```

    curr.next = prev
    prev = curr
    curr = nextNode
return prev

```

### 5.3 Sub-Range Reversal Template (positions left..right, 1-indexed)

```

function reverseBetween(head, left, right):
    dummy = new Node(0); dummy.next = head
    predecessor = dummy
    for i in range(1, left):
        predecessor = predecessor.next

    curr = predecessor.next
    prev = null
    for i in range(0, right - left + 1):
        nextNode = curr.next
        curr.next = prev
        prev = curr
        curr = nextNode

    predecessor.next.next = curr      # old sublist head now connects to remainder
    predecessor.next = prev          # predecessor connects to new sublist head
    return dummy.next

```

### 5.4 Group-of-K Reversal Template (recursive)

```

function reverseKGroup(head, k):
    node = head
    count = 0
    while node != null and count < k:
        node = node.next
        count = count + 1
    if count < k:
        return head      # fewer than k nodes left, leave as-is

    prev = reverseKGroup(node, k)      # recursively reverse the rest first
    curr = head
    for i in range(0, k):
        nextNode = curr.next
        curr.next = prev
        prev = curr
        curr = nextNode
    return prev

```

### 5.5 Interview Thinking Process

1. "I'll use three pointers — prev, curr, next — to flip links one at a time in O(1) space."
2. "If this is a sub-range or grouped reversal, I first need to locate the boundary nodes, ideally with a dummy node to avoid special-casing the head."
3. "I must save curr.next *before* overwriting it, or I lose the rest of the list — that's the one subtlety that causes most bugs."
4. "After reversing, I re-link the boundary: predecessor → new sublist head, old sublist head → remainder."
5. "Let me dry-run on a 4-5 node list to confirm the re-linking before finalizing."

---

## SECTION 6 — Time & Space Complexity

| Case         | Time                                                                           | Space                                            | Why                                                             |
|--------------|--------------------------------------------------------------------------------|--------------------------------------------------|-----------------------------------------------------------------|
| Worst Case   | $O(n)$                                                                         | $O(1)$ iterative / $O(n)$ recursive (call stack) | Every node visited exactly once                                 |
| Average Case | $O(n)$                                                                         | $O(1)$                                           | No dependency on data values                                    |
| Best Case    | $O(n)$ (must still traverse at least once)                                     | $O(1)$                                           | Even a “no-op” reversal ( $n=1$ ) needs one check               |
| Amortized    | $O(n)$ even for grouped reversal ( $k$ -group) across multiple recursive calls | $O(1)$ iterative, $O(n/k)$ recursive stack depth | Total nodes processed across all groups is still bounded by $n$ |

**Why recursive uses  $O(n)$  space:** each recursive call frame stays on the call stack until the base case is hit, effectively trading the  $O(1)$  iterative space for  $O(n)$  (or  $O(n/k)$  for grouped) call-stack space — an important trade-off to mention explicitly in interviews.

---

## SECTION 7 — Edge Cases

| Edge Case                                                           | Example                                      | Handling                                                                                |
|---------------------------------------------------------------------|----------------------------------------------|-----------------------------------------------------------------------------------------|
| Empty list                                                          | <code>null</code>                            | Return <code>null</code> immediately                                                    |
| Single node                                                         | <code>[5]</code>                             | Loop runs once, <code>prev</code> becomes the single node, correctly “reversed” (no-op) |
| Two nodes                                                           | <code>[1, 2]</code>                          | Verify link flips correctly: <code>2 → 1 → null</code>                                  |
| Reverse entire list ( <code>left=1</code> , <code>right=n</code> )  | Sub-range = full list                        | Dummy node handles this uniformly, no special-case needed                               |
| <code>left == right</code> (single-node sub-range)                  | No visible change expected                   | Loop runs once, re-linking should be a no-op                                            |
| <code>k</code> larger than remaining nodes ( <code>k</code> -group) | Leftover $< k$ nodes                         | Must leave remaining nodes <b>unreversed</b> — a common requirement misread             |
| Doubly linked list variant                                          | Need to also flip <code>prev</code> pointers | Must update both <code>next</code> and <code>prev</code> references, not just one       |
| Palindrome check with odd-length list                               | Middle node handling                         | Decide whether middle node participates in comparison (usually excluded)                |

**Common mistakes:** forgetting to save `curr.next` before overwriting (classic beginner bug — loses the rest of the list); off-by-one when counting `k` nodes for grouped reversal; forgetting to re-link the **old head of the reversed sublist** to the remainder (it becomes the new tail of that sublist).

---

## SECTION 8 — Pros & Cons

**Advantages:**  $O(1)$  space (iterative version); conceptually simple three-pointer loop; generalizes cleanly to sub-ranges and grouped variants. **Disadvantages:** Sub-range/grouped variants require careful boundary bookkeeping with dummy nodes — a common source of subtle bugs; recursive version trades space for elegance. **Trade-offs:** Iterative ( $O(1)$  space, slightly more verbose) vs. Recursive ( $O(n)$  stack space, cleaner code, especially for  $k$ -group reversal). **Limitations:** Doesn't apply to non-chain structures; reversing very large lists recursively risks stack overflow in languages without tail-call optimization. **Inefficient when:** N/A for its exact use case —  $O(n)$  is optimal since every node must be visited at least once to flip its pointer.

---

## SECTION 9 — Real World Applications

| Domain                                   | Application                                                                                             |
|------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Operating Systems                        | Reversing linked free-lists or LRU cache chains during certain maintenance operations                   |
| Databases                                | Reversing linked B+-tree leaf chains for backward range scans                                           |
| Version Control (Git internals)          | Conceptually related: reversing a commit history chain for certain traversal/display operations         |
| Browser History                          | "Back/forward" navigation stacks conceptually modeled as reversible chains                              |
| Text Editors                             | Undo/redo chain manipulation shares the same pointer-relinking mental model                             |
| Networking                               | Reordering/reversing linked packet buffers for certain protocol-level operations                        |
| Compilers                                | Reversing linked instruction lists during certain intermediate-representation optimization passes       |
| Interview Pipelines (Google/Amazon/Meta) | A canonical warm-up to assess pointer discipline before harder tree/graph pointer manipulation problems |

---

## SECTION 10 — Interview Strategy

**How seniors answer:** They immediately reach for a dummy node to eliminate head-of-list special-casing, verbalize the "save-next-before-overwrite" invariant explicitly, and offer both iterative ( $O(1)$  space) and recursive (cleaner for  $k$ -group) solutions, discussing the space trade-off unprompted.

**How juniors answer:** They often attempt reversal without a dummy node, leading to fragile special-case branches for the head; or they forget to save `curr.next`, causing a runtime error that they debug reactively instead of preventing proactively.

**Typical follow-ups:** "Can you reverse only between positions  $m$  and  $n$ ?" "Can you reverse in groups of  $k$ ?" "Can you do it recursively?" "What if it's a doubly linked list?"

**Optimization questions:** "Can you avoid the extra pass to count list length for  $k$ -group reversal?" (Discuss combining counting and reversal in a single traversal where feasible.)

---

## SECTION 11 — Pattern Variations

| Variation                      | Description                                    | Example                                       |
|--------------------------------|------------------------------------------------|-----------------------------------------------|
| Full Reversal                  | Reverse entire list                            | Reverse Linked List                           |
| Sub-Range Reversal             | Reverse only nodes between two positions       | Reverse Linked List II                        |
| Grouped Reversal (k at a time) | Reverse every consecutive group of k nodes     | Reverse Nodes in k-Group                      |
| Pairwise Swap                  | Special case of k=2 grouped reversal           | Swap Nodes in Pairs                           |
| Alternating Group Reversal     | Reverse every other group, leave others intact | Reverse Nodes in Even Length Groups (related) |
| Reversal + Merge               | Reverse second half, merge with first half     | Reorder List                                  |
| Doubly Linked List Reversal    | Flip both next and prev references             | Design Doubly Linked List variants            |

## SECTION 12 — Comparison With Other Patterns

| Pattern              | Difference                                                                   | When To Prefer                                                             |
|----------------------|------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Two Pointers (array) | Index-based swap on random-access structures                                 | Arrays/strings, not linked chains                                          |
| Fast & Slow Pointers | Finds structural properties (midpoint/cycle), doesn't itself rewire pointers | Combine with Reversal (e.g., Palindrome Linked List = midpoint + reversal) |
| Stack-Based Reversal | O(n) space, conceptually simpler                                             | When space isn't constrained or reversal order is complex                  |
| Recursion            | Elegant for grouped reversal, O(n) stack space                               | When code clarity matters more than strict O(1) space                      |

## SECTION 13 — Problem Classification

| Difficulty | Characteristics                                          | Examples                                          |
|------------|----------------------------------------------------------|---------------------------------------------------|
| Easy       | Full list reversal                                       | Reverse Linked List                               |
| Medium     | Sub-range or pairwise reversal with boundary bookkeeping | Reverse Linked List II, Swap Nodes in Pairs       |
| Hard       | Grouped reversal with recursion/iteration combination    | Reverse Nodes in k-Group                          |
| Very Hard  | Multi-phase combinations (reversal + merge + midpoint)   | Reorder List, Reverse Nodes in Even Length Groups |



## SECTION 14 — 30 Interview Problems

| #  | Problem                                 | Difficulty | Companies                       | Why Pattern Applies                                                  | Learning Objective                |
|----|-----------------------------------------|------------|---------------------------------|----------------------------------------------------------------------|-----------------------------------|
| 1  | Reverse Linked List                     | Easy       | Amazon, Meta, Microsoft, Google | Direct full reversal                                                 | Foundational three-pointer loop   |
| 2  | Reverse Linked List II                  | Medium     | Amazon, Meta, Microsoft         | Sub-range reversal with dummy node                                   | Boundary bookkeeping              |
| 3  | Swap Nodes in Pairs                     | Medium     | Amazon, Meta                    | k=2 grouped reversal                                                 | Grouped reversal basics           |
| 4  | Reverse Nodes in k-Group                | Hard       | Amazon, Meta, Microsoft, Google | General grouped reversal                                             | Recursive + iterative combination |
| 5  | Palindrome Linked List                  | Medium     | Meta, Amazon, Microsoft         | Midpoint (Fast/Slow) + reversal + comparison                         | Cross-pattern combination         |
| 6  | Reorder List                            | Medium     | Amazon, Meta                    | Midpoint + reversal + merge                                          | Multi-phase combination           |
| 7  | Add Two Numbers                         | Medium     | Amazon, Microsoft, Meta         | Related list manipulation (contrast: no reversal, uses carry logic)  | Boundary awareness                |
| 8  | Add Two Numbers II                      | Medium     | Amazon, Meta                    | Requires reversal to align digit order                               | Reversal as sub-routine           |
| 9  | Rotate List                             | Medium     | Amazon, Microsoft               | Related pointer manipulation (not reversal, but similar bookkeeping) | Pointer bookkeeping practice      |
| 10 | Odd Even Linked List                    | Medium     | Amazon, Microsoft               | Related in-place restructuring                                       | In-place chain restructuring      |
| 11 | Design Linked List                      | Medium     | Google, Amazon                  | Foundational pointer manipulation practice                           | Pointer discipline                |
| 12 | Flatten a Multilevel Doubly Linked List | Medium     | Meta, Microsoft                 | Related but different restructuring goal                             | Contrast with reversal            |

| #  | Problem                                | Difficulty | Companies               | Why Pattern Applies                                          | Learning Objective                  |
|----|----------------------------------------|------------|-------------------------|--------------------------------------------------------------|-------------------------------------|
| 13 | Reverse Nodes in Even Length Groups    | Medium     | Google                  | Alternating grouped reversal with variable group sizes       | Advanced grouped reversal           |
| 14 | Remove Duplicates from Sorted List     | Easy       | Amazon                  | Related in-place list manipulation                           | In-place manipulation practice      |
| 15 | Remove Duplicates from Sorted List II  | Medium     | Amazon, Microsoft       | Related but uses dummy node + deletion, not reversal         | Dummy node mastery                  |
| 16 | Merge Two Sorted Lists                 | Easy       | Amazon, Meta, Microsoft | Related pointer manipulation (contrast: merge, not reversal) | Pointer redirection practice        |
| 17 | Merge k Sorted Lists                   | Hard       | Google, Amazon, Meta    | Related (uses heap + merge, contrast case)                   | Recognizing pattern boundaries      |
| 18 | Partition List                         | Medium     | Amazon, Microsoft       | Related in-place restructuring via dummy nodes               | Dummy node + restructuring          |
| 19 | Copy List with Random Pointer          | Medium     | Amazon, Meta            | Related traversal (contrast: no reversal)                    | Pattern-boundary awareness          |
| 20 | Linked List Cycle II (as prerequisite) | Medium     | Amazon, Meta            | Combine with reversal in some palindrome variants            | Cross-pattern reinforcement         |
| 21 | Sort List                              | Medium     | Microsoft, Amazon       | Uses midpoint-finding + merge sort, related family           | Cross-pattern (merge sort on lists) |
| 22 | Split Linked List in Parts             | Medium     | Google                  | Related length-based traversal, not reversal itself          | Length-aware traversal              |

| #  | Problem                                           | Difficulty      | Companies                       | Why Pattern Applies                                           | Learning Objective                   |
|----|---------------------------------------------------|-----------------|---------------------------------|---------------------------------------------------------------|--------------------------------------|
| 23 | Delete Node in a Linked List                      | Easy            | Amazon, Meta                    | Related pointer manipulation without reversal                 | Contrast case                        |
| 24 | Convert Binary Number in a Linked List to Integer | Easy            | Amazon                          | Simple traversal contrast problem                             | Basic traversal reinforcement        |
| 25 | Design a Doubly Linked List with Reversal Support | Custom/Advanced | Google, Amazon                  | Doubly linked reversal variant                                | Bidirectional pointer flipping       |
| 26 | Reverse Only Letters (string variant, contrast)   | Easy            | Amazon                          | Contrast: Two Pointers on string, not linked list             | Pattern-boundary awareness           |
| 27 | LRU Cache                                         | Medium          | Amazon, Meta, Google, Microsoft | Uses doubly linked list manipulation (contrast, not reversal) | Doubly linked list pointer practice  |
| 28 | Design a Skiplist                                 | Hard            | Google                          | Related multi-level pointer manipulation (contrast)           | Advanced pointer structure awareness |
| 29 | Next Greater Node in Linked List                  | Medium          | Amazon                          | Contrast problem (uses monotonic stack, not reversal)         | Pattern-boundary awareness           |
| 30 | Reverse Nodes Between Zeroes                      | Medium          | Amazon (variant)                | Segmented reversal based on delimiter values                  | Custom boundary detection + reversal |

## SECTION 15 — Common Mistakes

1. Forgetting to save `curr.next` before reassigning `curr.next = prev` — loses the rest of the list irrecoverably. *Fix:* always save first, reassign second, advance third — in that strict order.
2. Not using a dummy node for sub-range/grouped reversal, leading to fragile special-casing when the reversal starts at the head. *Fix:* always prepend a dummy node when the head might change.
3. Forgetting to re-link the **old sublist head** (now the tail of the reversed segment) to the remainder of the list. *Fix:* explicitly track and connect this after the reversal loop.

4. Off-by-one errors when counting exactly  $k$  nodes for grouped reversal. *Fix:* count first in a separate small loop, verify `count == k` before reversing.
5. Confusing which pointer is the “new head” after reversal (it’s `prev`, not `curr`, since `curr` becomes `null` at loop exit). *Fix:* always return `prev`.

**Why people fail:** the logic is simple in isolation but combines multiple simultaneous pointer updates that must happen in a precise order — under interview pressure, candidates reorder statements incorrectly (e.g., reassigning before saving) and produce silently broken lists that are hard to debug live.

---

## SECTION 16 — Optimization Techniques

- **Time:** Already optimal at  $O(n)$  — focus on avoiding redundant traversals (e.g., don’t count list length separately if it can be combined with the reversal pass).
  - **Space:** Prefer iterative over recursive for very long lists to avoid stack overflow risk; iterative is the true  $O(1)$  space solution.
  - **Readability:** Use a dummy node consistently; name pointers descriptively (`prev`, `curr`, `next` — not `p`, `q`, `r`).
  - **Interview performance:** Draw the three-pointer state at each step on the whiteboard before coding — this visual trace prevents the most common ordering bugs.
- 

## SECTION 17 — Multiple Language Templates

### Java

```
public ListNode reverseList(ListNode head) {
    ListNode prev = null, curr = head;
    while (curr != null) {
        ListNode next = curr.next;
        curr.next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
```

### JavaScript

```
function reverseList(head) {
    let prev = null, curr = head;
    while (curr) {
        const next = curr.next;
        curr.next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
```

## PHP

```
function reverseList($head) {
    $prev = null; $curr = $head;
    while ($curr != null) {
        $next = $curr->next;
        $curr->next = $prev;
        $prev = $curr;
        $curr = $next;
    }
    return $prev;
}
```

## Python

```
def reverse_list(head):
    prev, curr = None, head
    while curr:
        next_node = curr.next
        curr.next = prev
        prev = curr
        curr = next_node
    return prev
```

## Go

```
func reverseList(head *ListNode) *ListNode {
    var prev *ListNode
    curr := head
    for curr != nil {
        next := curr.Next
        curr.Next = prev
        prev = curr
        curr = next
    }
    return prev
}
```

## C++

```
ListNode* reverseList(ListNode* head) {
    ListNode *prev = nullptr, *curr = head;
    while (curr) {
        ListNode* next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
```

## SECTION 18 — Dry Runs

### Small Input

1 → 2 → 3 → null

```
prev=null curr=1: next=2; 1.next=null; prev=1; curr=2
prev=1    curr=2: next=3; 2.next=1;   prev=2; curr=3
prev=2    curr=3: next=null; 3.next=2; prev=3; curr=null
Loop ends. Return prev=3 → 3 → 2 → 1 → null
```

### Large Input (Conceptual)

For a 100,000-node list, the single pass performs exactly 100,000 pointer flips — no revisits, no extra memory beyond three pointer variables, confirming true  $O(n)$  time /  $O(1)$  space regardless of list size.

### Corner Case

Empty list: `curr = head = null` → while loop never executes → return `prev = null` immediately, correctly representing an empty reversed list. Single node [5]: one iteration → `prev = node5`, `node5.next = null` (unchanged, correctly a no-op reversal).

---

## SECTION 19 — Advanced Concepts

- **Recursive reversal derivation:** `reverseList(head) = reverseList(head.next)` then fix `head.next.next = head`; `head.next = null` — the recursion reverses everything *after* the current node first, then attaches the current node to the end; understanding this derivation helps generalize to k-group recursion.
  - **Doubly linked list reversal:** requires swapping both `next` and `prev` on every node (not just one field), doubling the per-node work but remaining  $O(n)$  overall.
  - **Reversal as a building block:** many “hard” linked-list problems (Reorder List, Add Two Numbers II, Palindrome Linked List) are actually **easy problems in disguise** once decomposed into: locate boundary (often via Fast/Slow) → reverse a segment → merge/compare — recognizing this decomposition is a major interview differentiator.
  - **Interview hack:** always ask “should I return the new head, or should I modify in place and the caller already has a reference?” — clarifying this avoids a subtle but common miscommunication.
- 

## SECTION 20 — Senior Engineer Insights

Staff engineers view linked-list reversal not as an isolated trick but as the **canonical example of in-place pointer surgery** — the same discipline (save-before-overwrite, invariant-per-iteration reasoning) applies to far more complex structures: rebalancing trees in place, reversing segments of a doubly-linked free list in a custom allocator, or restructuring a directed acyclic dependency graph without allocating new nodes. Interviewers use this pattern to gauge whether a candidate can maintain multiple simultaneous invariants under pointer mutation without introducing use-after-free or dangling-reference bugs — a skill directly transferable to systems-level code review.

---

## SECTION 21 — Cheat Sheet

PATTERN: In-Place Linked List Reversal

RECOGNIZE: "reverse," "reverse between," "reverse in groups of k," O(1) space linked-list restructuring  
TEMPLATE:

```
prev = null; curr = head
while curr:
    next = curr.next      # SAVE FIRST
    curr.next = prev      # FLIP
    prev = curr; curr = next  # ADVANCE
return prev
```

COMPLEXITY: O(n) time, O(1) space (iterative) / O(n) space (recursive, call stack)

KEY PROOF: save-next-before-overwrite invariant preserves reachability to the untouched suffix at every step

WATCH FOR: dummy nodes for sub-range/grouped variants, re-linking old sublist head to remainder, order of operations

DOESN'T APPLY WHEN: need to preserve original + build a copy, non-chain structures

---

## SECTION 22 — Revision Notes (5-Minute Review)

- Three pointers: prev, curr, next — save next before overwriting curr.next.
  - New head after full reversal is prev, not curr (curr ends as null).
  - Sub-range/grouped reversal = boundary-finding + same core loop + re-linking via dummy node.
  - Recursive reversal trades O(1) space for O(n) call-stack space and cleaner k-group code.
  - Many “hard” list problems decompose into: Fast/Slow boundary-finding + Reversal + Merge/Compare.
  - Doubly linked lists need both next and prev flipped.
- 

## SECTION 23 — Practice Roadmap

| Stage        | Focus                                       | Recommended LeetCode Problems                                                            |
|--------------|---------------------------------------------|------------------------------------------------------------------------------------------|
| Beginner     | Full reversal mechanics                     | Reverse Linked List (206)                                                                |
| Intermediate | Sub-range and pairwise reversal             | Reverse Linked List II (92), Swap Nodes in Pairs (24)                                    |
| Advanced     | Grouped reversal, cross-pattern combination | Reverse Nodes in k-Group (25), Palindrome Linked List (234)                              |
| Expert       | Multi-phase restructuring                   | Reorder List (143), Add Two Numbers II (445), Reverse Nodes in Even Length Groups (2181) |

---

## SECTION 24 — Memory Tricks

- **Mnemonic:** “Save, Flip, Advance” (SFA) — Save next, Flip curr.next to prev, Advance both pointers.
  - **Visualization:** People holding hands who **switch handshake direction** as you walk down the line — no one moves position, only the grip direction changes.
  - **Recognition shortcut:** “Reverse” + “linked list” + “O(1) space” → this pattern, no further analysis needed.
-

## SECTION 25 — Final Summary

In-Place Linked List Reversal achieves  $O(1)$ -space pointer flipping by maintaining a simple but strict invariant: **always save the reference to what comes next before you overwrite the current node's pointer.** The single most important thing to remember forever: **the new head is prev, not curr, and every “hard” linked-list restructuring problem is almost always a composition of boundary-finding (often via Fast & Slow Pointers), this reversal primitive, and a final merge or comparison step.** Master the three-pointer loop once, and grouped/sub-range variants become straightforward bookkeeping exercises rather than new algorithms to learn from scratch.